Buổi 10

Cây và Cây nhị phân

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái niệm và biểu diễn cây

1.1 Định nghĩa cây
1.2 Một số khái niệm liên quan
1.3 Biểu diễn cây tổng quát

2. Cây nhị phân

2.1 Định nghĩa và tính chất cây nhị phân
2.2 Biểu diễn cây nhị phân
2.3 Các phép duyệt cây nhị phân
2.4 Minh họa các phép duyệt
2.5 Cây biểu thức – ứng dụng của các phép duyệt
2.6 Đếm nút, đếm lá, tính chiều cao
2.7 Tìm kiếm và duyệt theo mức
2.8 Duyệt không đệ quy và hủy cây
2.9 Khôi phục cây từ kết quả duyệt
2.10 Một số thao tác khác trên cây nhị phân

3. Tổng kết và thực hành

3.1 Tóm tắt và bài thực hành buổi 10
3.2 Câu hỏi và bài tập

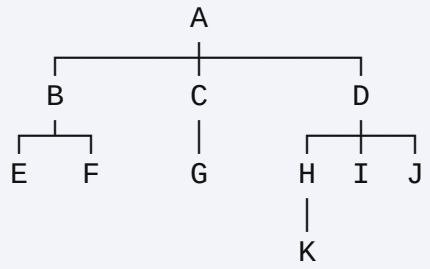
1.1 Định nghĩa cây

Định nghĩa (đệ quy)

Cây (tree) là một tập hữu hạn các **nút** cùng kiểu, có thể rỗng; nếu không rỗng thì có một nút đặc biệt gọi là **gốc (root)**, các nút còn lại được chia thành các tập rời nhau T_1, T_2, …, T_k, mỗi tập lại là một cây gọi là **cây con** của gốc.

- Cây là cấu trúc **phân cấp**, **không tuyến tính** (khác danh sách: mỗi nút có thể có nhiều nút "kế tiếp").
- Giữa hai nút bất kỳ có **đúng một đường đi**; cây có N nút thì có **N − 1** cạnh; không có chu trình.
- **Ví dụ thực tế:**
  - Cây thư mục của hệ điều hành.
  - Sơ đồ tổ chức (Hiệu trưởng → Khoa → Bộ môn).
  - Mục lục sách (Chương → Mục → Tiểu mục).
  - Cây gia phả, cây biểu thức số học, cây quyết định.

Ví dụ 10.1 – Cây T dùng xuyên suốt phần 1



- Gốc là A; A có 3 cây con với gốc B, C, D.
- Cây con gốc D lại có 3 cây con gốc H, I, J.
- 11 nút, 10 cạnh.

1.2 Một số khái niệm liên quan

Khái niệm	Định nghĩa	Trên cây T (ví dụ 10.1)
Bậc của nút	Số cây con của nút	bậc(A) = 3, bậc(B) = 2, bậc(C) = 1, bậc(H) = 1, bậc(E) = 0
Bậc của cây	Bậc lớn nhất của các nút	3 ⇒ T là cây **tam phân**
Nút lá	Nút có bậc 0	E, F, G, K, I, J
Nút nhánh (nút trong)	Nút có bậc > 0, không phải gốc	B, C, D, H
Cha – con	Gốc của cây con là con của gốc cây	D là cha của H, I, J; H là con của D
Anh em	Các nút cùng cha	H, I, J là anh em
Tổ tiên – hậu duệ	Các nút trên đường đi từ gốc tới nút	Tổ tiên của K: A, D, H
Mức (level)	Gốc có mức 1; con có mức = mức cha + 1	mức(A) = 1, mức(D) = 2, mức(H) = 3, mức(K) = **4**
Chiều cao (độ sâu)	Mức lớn nhất của các nút trong cây	h(T) = **4**
Độ dài đường đi	Số cạnh trên đường đi từ gốc tới nút	A → K: 3 cạnh
Rừng	Tập các cây rời nhau	Bỏ gốc A ⇒ rừng gồm 3 cây gốc B, C, D

Lưu ý quy ước

Một số tài liệu quy ước gốc có mức **0** (khi đó chiều cao T là 3). Môn học dùng quy ước của giáo trình: **gốc mức 1**, cây rỗng có chiều cao 0.

- **Cách 1 – mảng cha:** mỗi nút lưu chỉ số **nút cha** (gốc: −1).

Nút	A	B	C	D	E	F	G	H	I	J	K
Chỉ số	0	1	2	3	4	5	6	7	8	9	10
Cha	−1	0	0	0	1	1	2	3	3	3	7

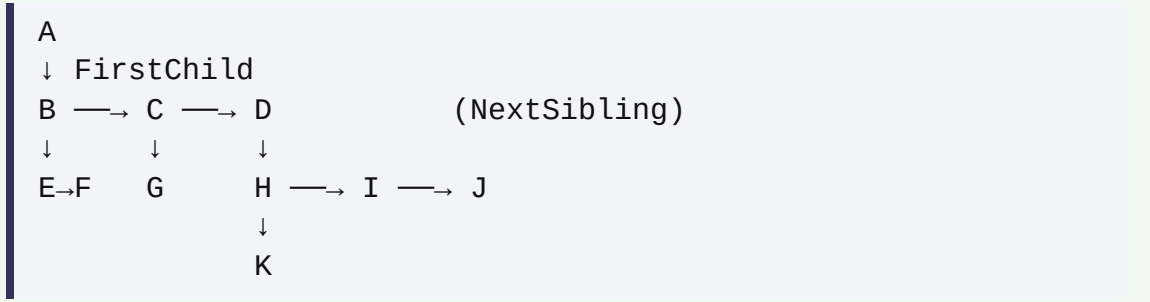
  - Tìm cha O(1); tìm các con phải duyệt toàn mảng O(N).
- **Cách 2 – mỗi nút có mảng con trỏ tới các con** (bậc tối đa m):

```
typedef struct TNode {
   T Key;
   TNode *Con[m];      // m = bậc của cây
} TNode;
```

  - Lãng phí: cây N nút có N·m con trỏ nhưng chỉ N − 1 con trỏ khác NULL.
- **Cách 3 – con đầu, em kế (first child – next sibling):** mỗi nút có 2 con trỏ: **con trái nhất** và **anh em kế bên phải**.

```
typedef struct GNode {
   T Key;
   GNode *FirstChild;   // con đầu tiên
   GNode *NextSibling;  // em kế
} GNode;
```

Ví dụ 10.2 – Cây T theo cách 3



- Mỗi nút chỉ cần **2 con trỏ** bất kể bậc ⇒ **mọi cây tổng quát** đều biểu diễn được bằng một **cây nhị phân** ⇒ lý do môn học tập trung vào cây nhị phân.

2.1 Định nghĩa và tính chất cây nhị phân

Định nghĩa

Cây nhị phân là cây mà mỗi nút có **tối đa 2** cây con, được phân biệt là **cây con trái** và **cây con phải** (có thứ tự).

Dạng đặc biệt	Mô tả
Cây nhị phân đúng (đầy đủ)	Mọi nút trong đều có đúng 2 con
Cây nhị phân hoàn chỉnh	Mọi mức (trừ mức cuối) đều đầy; mức cuối các nút dồn về bên trái – dạng của **heap** (buổi 4)
Cây nhị phân hoàn hảo	Mọi nút trong có 2 con và mọi lá cùng mức
Cây suy biến	Mỗi nút trong chỉ có 1 con ⇒ thực chất là một danh sách

- **Tính chất** (gốc mức 1):
  - Số nút tối đa ở mức i: **2^(i−1)**.
  - Cây chiều cao h có tối đa **2^h − 1** nút.
  - Cây có N nút có chiều cao tối thiểu $\lceil \log_2(N+1) \rceil$, tối đa **N** (suy biến).
  - Số lá = số nút có 2 con + 1.

Ví dụ 10.3

- h = 4 ⇒ tối đa $2^4 - 1 =$ **15** nút; mức 4 tối đa $2^3 = 8$ nút.
- N = 1.000.000 nút ⇒ chiều cao tối thiểu $\lceil \log_2(10^6 + 1) \rceil =$ **20**; nếu suy biến: $\mathbf{10^6}$.
- ⇒ Các thao tác đi từ gốc xuống lá tốn O(h): từ 20 bước đến 10^6 bước – lý do cần **cây cân bằng** (buổi 12).

2.2 Biểu diễn cây nhị phân

- **Bằng mảng** (phù hợp cây hoàn chỉnh): nút ở chỉ số i có con trái **2i + 1**, con phải **2i + 2**, cha **(i − 1)/2**.

Ví dụ 10.4

```
        A(0)
       /    \
    B(1)    C(2)
    /  \       \
 D(3) E(4)    F(6)
```

0	1	2	3	4	5	6
A	B	C	D	E	–	F

- Ô 5 trống ⇒ lãng phí; cây suy biến chiều cao h tốn 2^h − 1 ô cho h nút.

- **Bằng con trỏ (theo giáo trình)** – cách tổng quát:

```
typedef struct BinT_Node
{  T Key;
   BinT_Node *BinT_Left;    // con trái
   BinT_Node *BinT_Right;   // con phải
} BinT_OneNode;
typedef BinT_OneNode *BinT_Type;
BinT_Type BinT_Root;        // quản lý cây
```

- **Khởi tạo:** `BinT_Root = NULL;` (cây rỗng)
- **Tạo nút mới:**

```
BinT_Type BinT_Create_Node(T NewData)
{  BinT_Type Pnode = new BinT_OneNode;
   if (Pnode != NULL)
   {  Pnode->BinT_Left  = NULL;
      Pnode->BinT_Right = NULL;
      Pnode->Key = NewData;
   }
   return (Pnode);
}
```

2.3 Các phép duyệt cây nhị phân

- **Duyệt cây:** thăm mỗi nút **đúng một lần**. Ký hiệu: **N** – thăm nút gốc (Node), **L** – duyệt cây con trái, **R** – duyệt cây con phải.

Phép duyệt	Thứ tự	Tên gọi
NLR	Gốc → Trái → Phải	Tiền tự (PreOrder)
LNR	Trái → Gốc → Phải	Trung tự (InOrder)
LRN	Trái → Phải → Gốc	Hậu tự (PostOrder)

- Các biến thể NRL, RNL, RLN là đối xứng (duyệt phải trước).

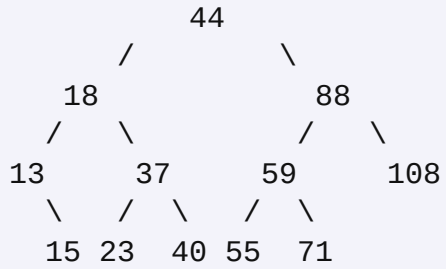
```
void BinT_PreOrder(BinT_Type Root)     // NLR
{  if (Root == NULL) return;
   printf("%4d", Root->Key);           // N
   BinT_PreOrder(Root->BinT_Left);     // L
   BinT_PreOrder(Root->BinT_Right);    // R
}
```

```
void BinT_InOrder(BinT_Type Root)      // LNR
{  if (Root == NULL) return;
   BinT_InOrder(Root->BinT_Left);      // L
   printf("%4d", Root->Key);           // N
   BinT_InOrder(Root->BinT_Right);     // R
}
void BinT_PostOrder(BinT_Type Root)    // LRN
{  if (Root == NULL) return;
   BinT_PostOrder(Root->BinT_Left);    // L
   BinT_PostOrder(Root->BinT_Right);   // R
   printf("%4d", Root->Key);           // N
}
```

- Ba hàm chỉ khác nhau ở **vị trí của lệnh thăm nút** (N).
- Mỗi nút được thăm một lần ⇒ **O(N)**; bộ nhớ ngăn xếp đệ quy O(h).

2.4 Minh họa các phép duyệt

Ví dụ 10.5 – Cây nhị phân B dùng xuyên suốt buổi 10, 11



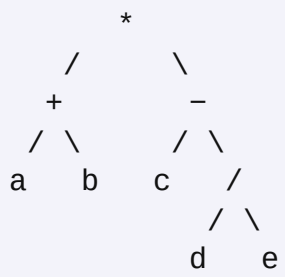
Phép duyệt	Kết quả
NLR (PreOrder)	44 18 13 15 37 23 40 88 59 55 71 108
LNR (InOrder)	13 15 18 23 37 40 44 55 59 71 88 108
LRN (PostOrder)	15 13 23 40 37 18 55 71 59 108 88 44
Theo mức	44 18 88 13 37 59 108 15 23 40 55 71

- **Phân tích NLR từng bước:** thăm 44 → sang cây trái gốc 18: thăm 18 → cây trái gốc 13: thăm 13, cây trái rỗng, cây phải gốc 15: thăm 15 → quay lên, duyệt cây phải của 18 (gốc 37): thăm 37, 23, 40 → quay lên gốc 44, duyệt cây phải gốc 88: 88, 59, 55, 71, 108.
- **Nhận xét quan trọng:** kết quả duyệt **LNR tăng dần** – vì B là **cây nhị phân tìm kiếm** (buổi 11).
- **Mẹo kiểm tra:** NLR luôn bắt đầu bằng **gốc**; LRN luôn kết thúc bằng **gốc**; nút lá trái nhất luôn đứng đầu kết quả LNR.

2.5 Cây biểu thức – ứng dụng của các phép duyệt

- **Cây biểu thức:** nút lá là **toán hạng**, nút trong là **toán tử**; cây con trái/phải là toán hạng trái/phải.

Ví dụ 10.6 – Cây của biểu thức (a + b) * (c − d / e)



Phép duyệt	Kết quả	Ý nghĩa
NLR	* + a b − c / d e	Biểu thức **tiền tố** (Ba Lan)
LNR	a + b * c − d / e	Trung tố (**mất ngoặc** – cần thêm ngoặc khi in)
LRN	a b + c d e / − *	Biểu thức **hậu tố** – tính bằng stack (buổi 9)

- **Tính giá trị** bằng duyệt LRN: tính hai cây con trước rồi áp toán tử ở gốc:

```
double TinhCay(ENode *p)
{  if (p->Left == NULL && p->Right == NULL)
      return p->GiaTri;                // lá: toán hạng
   double a = TinhCay(p->Left), b = TinhCay(p->Right);
   switch (p->Op) { case '+': return a + b;  case '-': return
a - b;
                    case '*': return a * b;  case '/': return
a / b; }
   return 0;
}
```

- a = 2, b = 3, c = 9, d = 8, e = 2 ⇒ (2 + 3) × (9 − 4) = **25**.

2.6 Đếm nút, đếm lá, tính chiều cao

- Hầu hết các thao tác trên cây được viết **đệ quy** theo mẫu: xử lý trường hợp **cây rỗng**, rồi kết hợp kết quả của **cây con trái** và **cây con phải**.

```
int BinT_Num_Node(BinT_Type Root)          // số nút
{  if (Root == NULL) return 0;
   return 1 + BinT_Num_Node(Root->BinT_Left)
            + BinT_Num_Node(Root->BinT_Right);
}
int BinT_Num_Leaf(BinT_Type Root)          // số lá
{  if (Root == NULL) return 0;
   if (Root->BinT_Left == NULL &&
       Root->BinT_Right == NULL) return 1;
   return BinT_Num_Leaf(Root->BinT_Left)
        + BinT_Num_Leaf(Root->BinT_Right);
}
int BinT_Height(BinT_Type Root)            // chiều cao
{  if (Root == NULL) return 0;
   int HL = BinT_Height(Root->BinT_Left);
   int HR = BinT_Height(Root->BinT_Right);
   return 1 + (HL > HR ? HL : HR);
}
```

Ví dụ 10.7 – Áp dụng trên cây B (ví dụ 10.5)

Hàm	Kết quả	Giải thích
Num_Node	**12**	1 (gốc) + 6 (cây trái gốc 18) + 5 (cây phải gốc 88)
Num_Leaf	**6**	15, 23, 40, 55, 71, 108
Height	**4**	Nhánh 44 → 18 → 13 → 15

- Chiều cao cây con gốc 18: h(13) = 2, h(37) = 2 ⇒ 1 + 2 = 3; cây gốc 88: h(59) = 2, h(108) = 1 ⇒ 3. Gốc: 1 + max(3, 3) = **4**.
- Kiểm tra tính chất: số lá 6 = số nút 2 con (44, 18, 37, 88, 59) + 1 = 6 ✓.

- Tương tự: tổng giá trị các nút, đếm nút có 1 con, tìm giá trị lớn nhất – đều O(N).

2.7 Tìm kiếm và duyệt theo mức

- **Tìm nút có Key = X** trên cây nhị phân **thường** (không có thứ tự) ⇒ phải xét cả hai cây con:

```
BinT_Type BinT_Search(BinT_Type Root, T X)
{  if (Root == NULL || Root->Key == X)
      return Root;
   BinT_Type p = BinT_Search(Root->BinT_Left, X);
   if (p != NULL) return p;
   return BinT_Search(Root->BinT_Right, X);
}
```

- Xấu nhất phải thăm **mọi nút** ⇒ O(N). Buổi 11: cây nhị phân **tìm kiếm** chỉ cần đi **một nhánh** ⇒ O(h).

- **Duyệt theo mức (BFS)** – thăm lần lượt mức 1, 2, 3, … từ trái sang phải; dùng **hàng đợi** (buổi 9):

```
void BinT_LevelOrder(BinT_Type Root)
{  BinT_Type Q[MaxLen];  int Front = 0, Rear = 0;
   if (Root != NULL) Q[Rear++] = Root;
   while (Front < Rear)
   {  BinT_Type p = Q[Front++];              // Get
      printf("%4d", p->Key);
      if (p->BinT_Left  != NULL) Q[Rear++] = p->BinT_Left;
// Add
      if (p->BinT_Right != NULL) Q[Rear++] = p->BinT_Right;
   }
}
```

Ví dụ 10.8 – Hàng đợi khi duyệt cây B

Q: [44] → lấy 44, thêm 18, 88 → lấy 18, thêm 13, 37 → lấy 88, thêm 59, 108 → … ⇒ **44 18 88 13 37 59 108 15 23 40 55 71**.

2.8 Duyệt không đệ quy và hủy cây

- **NLR không đệ quy bằng stack** (khử đệ quy – buổi 9):

```
void BinT_PreOrder_NR(BinT_Type Root)
{  BinT_Type S[MaxLen];  int Top = -1;
   if (Root != NULL) S[++Top] = Root;
   while (Top >= 0)
   {  BinT_Type p = S[Top--];              // Pop
      printf("%4d", p->Key);
      if (p->BinT_Right != NULL) S[++Top] = p->BinT_Right;
      if (p->BinT_Left  != NULL) S[++Top] = p->BinT_Left;
   }
}
```

- Push con **phải trước**, con **trái sau** ⇒ con trái ở đỉnh, được thăm trước ⇒ đúng thứ tự NLR.
- So sánh với duyệt theo mức: **cùng khung**, chỉ thay **hàng đợi** bằng **ngăn xếp** (BFS ↔ DFS).

- **Hủy toàn bộ cây** – phải dùng **LRN**: hủy hai cây con trước, hủy gốc sau cùng.

```
void BinT_Delete(BinT_Type &Root)
{  if (Root == NULL) return;
   BinT_Delete(Root->BinT_Left);
   BinT_Delete(Root->BinT_Right);
   delete Root;
   Root = NULL;
}
```

Vì sao không dùng NLR để hủy?

Nếu `delete Root` trước thì Root->BinT_Left, Root->BinT_Right đọc từ vùng nhớ **đã giải phóng** ⇒ lỗi. Nếu chỉ gán `Root = NULL` ⇒ rò rỉ toàn bộ cây.

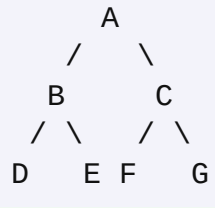
- **Sao chép cây** thì dùng NLR: tạo nút gốc mới trước, rồi sao chép hai cây con.

2.9 Khôi phục cây từ kết quả duyệt

- **Một** kết quả duyệt không xác định duy nhất cây; nhưng **NLR + LNR** (hoặc LRN + LNR) thì xác định **duy nhất** cây nhị phân (các Key khác nhau).
- **Ý tưởng:** phần tử **đầu** của NLR là **gốc**; tìm gốc trong LNR ⇒ bên trái gốc là các nút của **cây con trái**, bên phải là **cây con phải**; lặp lại đệ quy.

Ví dụ 10.9 – NLR = A B D E C F G, LNR = D B E A F C G

Bước	Gốc (từ NLR)	LNR trái	LNR phải
1	**A**	D B E	F C G
2 – cây trái	**B** (phần tử kế trong NLR)	D	E
3 – cây phải	**C**	F	G



⇒ LRN = **D E B F G C A**.

```
BinT_Type DungCay(char Pre[], char In[],
                  int &k, int L, int R)
{  // Pre[k]: gốc hiện tại; In[L..R]: các nút của cây
   if (L > R) return NULL;
   BinT_Type p = BinT_Create_Node(Pre[k++]);
   int m = L;
   while (In[m] != p->Key) m++;           // vị trí gốc trong
LNR
   p->BinT_Left  = DungCay(Pre, In, k, L, m - 1);
   p->BinT_Right = DungCay(Pre, In, k, m + 1, R);
   return p;
}
// Gọi: int k = 0; Root = DungCay(Pre, In, k, 0, N - 1);
```

- **Phản ví dụ:** NLR = A B và LRN = B A ứng với **hai** cây: B là con trái của A, hoặc B là con phải của A ⇒ NLR + LRN không đủ xác định cây.

2.10 Một số thao tác khác trên cây nhị phân

```
// Sao chép cây (NLR)
BinT_Type BinT_Copy(BinT_Type Root)
{  if (Root == NULL) return NULL;
   BinT_Type p = BinT_Create_Node(Root->Key);
   p->BinT_Left  = BinT_Copy(Root->BinT_Left);
   p->BinT_Right = BinT_Copy(Root->BinT_Right);
   return p;
}
// Hai cây giống nhau?
int BinT_Equal(BinT_Type A, BinT_Type B)
{  if (A == NULL || B == NULL) return A == B;
   return A->Key == B->Key
       && BinT_Equal(A->BinT_Left,  B->BinT_Left)
       && BinT_Equal(A->BinT_Right, B->BinT_Right);
}
// Mức của nút có Key = X (0 nếu không có)
int BinT_Level(BinT_Type Root, T X, int Muc)
{  if (Root == NULL) return 0;
   if (Root->Key == X) return Muc;
   int k = BinT_Level(Root->BinT_Left, X, Muc + 1);
   if (k != 0) return k;
   return BinT_Level(Root->BinT_Right, X, Muc + 1);
}
// Gọi: BinT_Level(Root, X, 1)
```

Ví dụ 10.10 – Áp dụng trên cây B

- BinT_Level(B, 71, 1) = **4** (44 → 88 → 59 → 71); BinT_Level(B, 99, 1) = 0.
- Đếm nút ở mức 3: 13, 37, 59, 108 ⇒ **4** nút.
- Cây gương của B: đổi chỗ cây con trái/phải tại mọi nút ⇒ duyệt LNR được dãy **giảm dần** 108 88 71 … 13.

Thao tác	Phép duyệt phù hợp	Lý do
In, tìm, đếm	Bất kỳ	Chỉ cần thăm mỗi nút một lần
Sao chép cây	NLR	Phải có nút cha trước khi gắn con
Hủy cây, tính chiều cao, tính biểu thức	LRN	Cần kết quả hai cây con trước
In tăng dần (cây BST)	LNR	Trái < gốc < phải
In theo mức	BFS (hàng đợi)	Theo khoảng cách tới gốc

3.1 Tóm tắt và bài thực hành buổi 10

- Cây: gốc, cây con, bậc, lá, mức, chiều cao; biểu diễn bằng mảng cha, mảng con trỏ con, con đầu – em kế (mọi cây ⇒ cây nhị phân).
- Cây nhị phân: tối đa 2 con; biểu diễn mảng (2i + 1, 2i + 2) hoặc con trỏ (BinT_Left, BinT_Right).
- Duyệt NLR, LNR, LRN (đệ quy), theo mức (hàng đợi), không đệ quy (ngăn xếp); các hàm đếm/chiều cao viết đệ quy theo mẫu "rỗng – trái – phải".

Thực hành – Cài đặt cây nhị phân và các phép duyệt đệ quy

- 1) Cài đặt cấu trúc BinT_Type, hàm tạo nút; **xây dựng thủ công** cây B (ví dụ 10.5) bằng các lệnh gán con trỏ.
- 2) Cài đặt NLR, LNR, LRN, duyệt theo mức; đối chiếu kết quả với bảng ở ví dụ 10.5.
- 3) Cài đặt: đếm nút, đếm lá, đếm nút có đúng 1 con, chiều cao, tổng các Key, Key lớn nhất, mức của nút có Key = X.
- 4) In cây **dạng thụt lề** (mỗi nút một dòng, thụt vào theo mức) để quan sát cấu trúc:

```
void InCay(BinT_Type Root, int Muc)
{  if (Root == NULL) return;
   InCay(Root->BinT_Right, Muc + 1);
   printf("%*s%d\n", 4 * Muc, "", Root->Key);
   InCay(Root->BinT_Left, Muc + 1);
}
```

- 5) Hủy cây khi kết thúc; kiểm tra rò rỉ bộ nhớ.

3.2 Câu hỏi và bài tập

Câu hỏi

- 1) Nêu định nghĩa đệ quy của cây; giải thích các khái niệm bậc, mức, chiều cao, lá.
- 2) Vì sao mọi cây tổng quát đều có thể biểu diễn bằng cây nhị phân?
- 3) Ba hàm duyệt NLR, LNR, LRN khác nhau ở điểm nào? Hủy cây phải dùng phép duyệt nào? Vì sao?

Bài tập (có đáp số)

- 1) Cho cây nhị phân có NLR = **A B D E C F G** và LNR = **D B E A F C G**. Vẽ cây và cho kết quả LRN. → Gốc A; cây trái {D, B, E} gốc B (con D, E); cây phải {F, C, G} gốc C (con F, G) ⇒ LRN = **D E B F G C A**.
- 2) Vẽ cây biểu thức của `a * (b − c) + d / e` và viết biểu thức tiền tố, hậu tố. → Tiền tố **+ * a − b c / d e**; hậu tố **a b c − * d e / +**.
- 3) Viết hàm kiểm tra hai cây nhị phân có **giống nhau** (cùng cấu trúc và giá trị) không; hàm tạo **cây đối xứng** (gương) của một cây.
- 4) Viết hàm đếm số nút ở mức k; in các nút trên đường đi dài nhất từ gốc đến lá.